COMP1000 Unix & C Programming

Assignment Short Report

Labyrinth Game - C89 Implementation

Semester 2 2026

Uvindu Randula Gunathilake
23610903

1. Program Overview

This report explains how the Labyrinth game was built and organized using the C programming language. In this terminal-based game, players navigate a maze to find treasure and reach the goal while avoiding a moving enemy. The project is split into several files, with each part handling a specific job to keep the code neat and manageable.

2. File Structure and Responsibilities

To make the program easier to understand and maintain, it is divided into different files. Each file has a clear purpose, as described in the table below.

File	Header	Responsibility
main.c	—	The starting point of the game. It controls the main loop, processes player moves, saves the game state, and cleans up memory when the game ends.
game.c	game.h	Manages the main logic of each turn. It checks if the player has won by reaching the goal or lost by running into the enemy.
map.c	map.h	Sets up the game board from a file. It handles memory for the map, prints it to the screen with colors, and clears the memory afterward.
player.c	player.h	Controls how the player moves in four directions. It ensures players don't walk through walls or leave the map, and detects when treasure is found.
enemy.c	enemy.h	Runs the enemy's behavior. It decides where the enemy moves, how it turns at intersections, and speeds it up once the treasure is taken.
linked_list.c	linked_list.h	A list that stores past game moments. This allows the "undo" feature to work by letting players step back to an earlier version of the game.
terminal.c	terminal.h	Adjusts terminal settings so the game can read single key presses instantly without the player needing to press "Enter."

random.c	random.h	Generates random numbers so the enemy makes unpredictable choices when it reaches a crossroads in the maze.

3. Functionality Descriptions

3.1 Command-Line Arguments Verification

File: main.c — Topic: Starting the Program

When you start the game, it checks if you provided a map file. If the file is missing or cannot be opened, the program displays a help message and closes safely without crashing.

```
if (argc != 2) { printf("Usage: ./labyrinth <map_file>\n"); result = 1; }
```

3.2 File I/O and 2D Map Initialisation

File: map.c — Topic: Loading the Game Board

The program reads the map file to find its size and then creates two versions of the grid in the computer's memory:

- A visual grid that stores characters like 'P' for player and 'T' for treasure.
- A logical grid that uses numbers to help the computer understand where walls and goals are located.

Each integer in the file is converted to the appropriate display character via convertToChar(). Special cell types (4 = player, 5 = enemy, 3 = treasure, 2 = goal) are recorded into the Map struct (playerRow, playerCol, enemyRow, enemyCol, etc.) during parsing.

3.3 Screen Clearing and Map Printing with Colours

File: map.c — Topic: Displaying the Game

To make the game look smooth, the map is redrawn from the top-left corner instead of clearing the whole screen, which prevents flickering. Different colors are used to help objects stand out:

Object	Condition (type[][])	Colour Applied
Wall	type == 1	White background
Goal	type == 2	Green background
Treasure	grid == 'T'	Yellow background
Player	grid == 'P'	Blue foreground
Enemy (normal)	type == 5, treasure = 0	Red foreground
Enemy (furious)	type == 5, treasure = 1	Red background, white foreground

3.4 Player Movement

File: player.c — Topic: Moving the Character

When a player chooses a direction, the game checks if the move is allowed. A move is blocked if:

- The player would walk off the edge of the map.
- The path is blocked by a wall.
- The player tries to enter the goal without having the treasure first.

If the move is safe, the player is placed in the new spot. If they step on a treasure, the game records that it has been collected. Only successful moves can be "undone" later.

3.5 Win and Lose Conditions

File: game.c — Topic: Winning and Losing

After every move, the game checks for three possible outcomes:

- Win: The player has the treasure and reaches the goal.
- Lose: The player and the enemy end up in the same spot.
- Continue: Neither of the above has happened yet.

The check is performed twice per turn in main.c — once after the player moves, and once after the enemy moves — so the game ends immediately when either event occurs.

3.6 Enemy Movement Algorithm

File: enemy.c — Topic: How the Enemy Moves

The enemy follows a set of rules to navigate the maze:

3.6.1 Direction Resolution

The enemy looks at its current direction to see which way is forward. It identifies if the paths ahead, to the left, or to the right are blocked by walls, treasures, or the goal.

3.6.2 Movement Cases

The enemy decides where to go based on its surroundings:

- If only forward is clear, it continues straight.
- At a crossroads, it randomly picks a clear path and turns if needed.
- Case 3 — Forward blocked, both sides free: RNG chooses left or right.
- If forward is blocked but a side is open, it turns and moves that way.
- In a dead end, it turns around completely and goes back.

3.6.3 Furious State

The enemy gets faster once the treasure is taken. Usually, it moves 2 steps per turn, but it moves 3 steps per turn in "furious" mode. There is a tiny pause between steps so you can see it moving.

- Normal state (treasureCollected == 0): enemy moves 2 steps.
- Furious state (treasureCollected == 1): enemy moves 3 steps.

Each step calls movement() and then isPlayerHit(), which checks if the enemy has reached the player. If the player has already been caught, the subsequent movement steps are skipped to prevent the enemy from moving through the player. A small delay of 0.13 seconds (newSleep()) is added between each step for visual effect.

3.7 UNDO Feature with Generic Linked List

Files: linked_list.c — Topic: Stepping Back in Time

3.7.1 Generic Linked List Design

The game uses a list to save "snapshots" of the board. These snapshots store the positions of the player and enemy, the direction the enemy is facing, and whether the treasure was found.

```
typedef struct Node { void* data; struct Node* next; } Node;
```

The data pointer is cast to GameState* wherever specific game state fields are needed. This means the list itself contains no game-specific logic — it simply stores and retrieves opaque pointers, satisfying the generic linked list requirement.

3.7.2 GameState Snapshot

GameState stores a complete snapshot of the game at a given moment:

- playerRow, playerCol — player position
- enemyRow, enemyCol — enemy position
- enemyFacing — the direction character of the enemy
- treasureCollected — flag for treasure state
- grid — a deep-copied char** of the full display grid
- type — a deep-copied int** of the full logical type grid

Both 2D arrays are allocated with malloc() inside saveState() and individually freed in restoreState() and freeList().

3.7.3 Stack Behaviour

This works like a stack of photos. Every move adds a new photo to the top. If you press undo, the game takes the top photo away and resets the game to how things looked in the previous one.

removeLastSave() is called when a player move is invalid — it discards the most recently pushed node without applying it, keeping the stack consistent.

4. Memory Management

The program is careful with the computer's memory. It only uses what it needs when it needs it and makes sure to release everything once the game is over to prevent any waste.

Allocation	Location	Freed In
Map struct + grid + type	createMap() in map.c	freeMap() in map.c
LinkedList struct	createList() in linked_list.c	freeList() in linked_list.c
Node structs	saveState() in linked_list.c	restoreState() / removeLastSave() / freeList()
GameState struct	saveState() in linked_list.c	restoreState() / removeLastSave() / freeList()
Deep-copy grid rows (char*)	saveState() in linked_list.c	Per-row free loop in restoreState() / freeList()
Deep-copy type rows (int*)	saveState() in linked_list.c	Per-row free loop in restoreState() / freeList()

freeList() is always called before freeMap() in main.c to ensure that all undo snapshots are released first. No global variables are used anywhere in the program.

5. Coding Standards Compliance

The code follows strict rules to ensure it is clean and professional:

- Data is shared safely between functions rather than being left out in the open.
- The game always finishes naturally without being forced to stop abruptly.
- No break outside switch — the game loop uses a while condition instead of break.
- No continue — not used anywhere in the codebase.
- No goto — not used.
- Functions are written to be simple, each finishing its task and returning to the main program in a predictable way.
- No #include of .c files — only header (.h) files are included.
- C89-compatible comments — only /* ... */ style comments are used throughout.
- Header guards — all .h files use #ifndef / #define / #endif guards.

6. Makefile

A special file is used to build the game. It tells the computer how to glue all the different code files together into one playable program while checking for any mistakes or warnings.

- CC — compiler variable (gcc)
- CFLAGS — compiler flags (-Wall -ansi -pedantic)
- OBJ — list of object files for all modules
- labyrinth: — main build target linking all object files
- Individual .o: — targets for each source file with correct prerequisites
- clean: — removes all .o files and the labyrinth executable

The makefile was written manually and does not rely on any IDE-generated output.

7. Known Limitations / Notes

- The enemy currently moves a set distance each turn to keep the challenge consistent with the game rules.
- The way the screen refreshes is designed for standard Linux systems; other systems might see a slight flicker on very big maps.

- The game assumes the map files provided are correctly set up with one player, one enemy, one goal, and one treasure.

8. References

- UCP Supplementary Video — Terminal raw mode (disableBuffer / enableBuffer in terminal.c)
- UCP Supplementary Video — newSleep() implementation using nanosleep() (enemy.c)
- UCP Supplementary Video — tput cup usage for screen refresh (map.c)
- GeeksForGeeks — fscanf file reading pattern (map.c, createMap())